

Angular Concepts Explained

From Absolute Basics to the Advanced Concepts Used in StreamCast
With Simple Language, Real-World Analogies & Project Examples

For: Afrin Banua | Project: streamcast-ui | Date: 19 May 2026

40 concepts | Foundation → Architecture → Forms → Communication → Modern Angular

Table of Contents

1. What is Angular?
2. Single-Page Application (SPA)
3. TypeScript
4. Decorators
5. Components
6. Selector
7. Modules vs Standalone Components
8. Template
9. Data Binding — The 4 Types
10. Directives
11. Pipes
12. Services
13. Dependency Injection (DI)
14. Lifecycle Hooks
15. Routing
16. Route Parameters
17. Lazy Loading
18. Guards
19. HttpClient
20. Observables
21. RxJS Operators
22. Subjects (advanced Observables)
23. Forms — Reactive vs Template-driven
24. Validators
25. @Input — Parent → Child
26. @Output + EventEmitter — Child → Parent
27. Shared Service — Sibling Communication
28. ng-content (Content Projection)
29. ng-template & ng-container
30. Signals (Angular 17 Reactivity)
31. computed() — Derived Signals
32. effect() — Side Effects on Signal Changes
33. Interceptors
34. Change Detection (CD)
35. The async pipe
36. Environment Configuration
37. AOT vs JIT
38. Tree-shaking
39. The inject() function
40. Provider Functions (provideX)

Part 1 — Foundation

1 What is Angular?

Simple words: Angular is a complete toolkit for building web apps. You write components (small UI pieces), and Angular wires them together and shows them in the browser.

Analogy: Angular is like **IKEA**. They give you the parts (components), the screws (services), the manual (router), and the toolbox (CLI). You assemble a kitchen (app). Without IKEA, you'd cut wood yourself.

Why use it instead of plain HTML+JS?

- **Plain JS:** you manually do `document.querySelector(...)`, `element.textContent = ...` for every change. Painful at scale.
- **Angular:** you describe *what* the UI should look like for any data, and Angular updates the DOM for you.

2 Single-Page Application (SPA)

Simple words: Old websites loaded a fresh HTML page on every click. SPAs load **one** HTML page, then JavaScript swaps parts of the screen.

Analogy: Old sites = **stage shows** (curtain drops between scenes). SPAs = **movies** (cuts happen seamlessly, no reload).

In StreamCast: Look at `index.html` — there's only one. Click "Titles" → URL changes to `/titles`, content swaps, but the page never reloads.

```
<!-- src/index.html — the ONLY html file -->
<body>
  <sc-root></sc-root>  <!-- Angular fills this in -->
</body>
```

3 TypeScript

Simple words: JavaScript + types. You declare what shape your data has, and the compiler catches mistakes before you run the app.

Analogy: Plain JS is like writing on a **blank paper** — you can write anything, even nonsense. TypeScript is **printed form fields** — "name", "date", "amount" each have a fixed type, so you can't put "hello" in the amount box.

```
// core/models/user.ts
export interface CurrentUser {
  email: string;      // must be a string
  role: Role;         // must be one of the 8 allowed roles
  token: string;
  expiresAt: number;  // must be a number
}
```

If you try `user.expiresAt = "tomorrow"`, TypeScript instantly flags it red.

4 Decorators

Simple words: A `@SomeName` annotation that adds metadata or extra behavior to a class.

Analogy: Decorators are like **labels on shipping boxes** — "FRAGILE", "THIS WAY UP". The box (class) is unchanged, but the label tells handlers (Angular) how to treat it.

Examples used:

- `@Component` — "this class is a UI piece"
- `@Injectable` — "this class can be injected as a dependency"
- `@Input` — "this property receives data from parent"
- `@Output` — "this property emits events upward"

5 Components

Simple words: A reusable UI piece — HTML + CSS + a TypeScript class that controls it.

Analogy: A component is a **Lego brick**. You build a castle (page) from many bricks. The same "tower" brick can be used in many castles.

```

@Component({
  selector: 'sc-login',           // ← HTML tag name → <sc-login>
  standalone: true,              // ← doesn't need a module
  imports: [CommonModule, ReactiveFormsModule],
  template: `<form>...</form>`, // ← what to render
})
export class LoginComponent {    // ← the brain
  loading = signal(false);
  submit() { ... }
}

```

Real-world parallel: Like the "Add to Cart" button on Amazon — it appears on the product page, the home page, the search page. Same component, same behavior, reused everywhere.

6 Selector

Simple words: The custom HTML tag name your component answers to.

Analogy: Like a **person's name** — call "Alice!" and Alice responds. Write `<sc-login>` and Angular puts the `LoginComponent` there.

```

<!-- somewhere in a template -->
<sc-login></sc-login>

```

StreamCast uses prefix `sc-` (StreamCast) for all its components to avoid name clashes with Material or Angular ones.

7 Modules vs Standalone Components

Old way (NgModule): You had to declare every component inside a module (`AppModule`). Like having to enroll every student in a department.

New way (standalone): Each component declares its own imports. Like a freelancer who brings their own tools.

StreamCast uses standalone everywhere — there is no AppModule at all.

```
// Modern standalone
@Component({
  standalone: true,
  imports: [CommonModule, RouterLink, MatIconModule],
  template: ...
})

// Bootstrap in main.ts
bootstrapApplication(AppComponent, appConfig); // NOT bootstrapModule
```

Why this is better:

- Less boilerplate
- Each component is portable — copy to another project and it works
- Lazy loading is one line: `loadComponent: () => import('...')`

8 Template

Simple words: The HTML part of a component, with Angular's special syntax mixed in.

Analogy: A template is a **fill-in-the-blank form letter**. "Dear ___, your order ___ is ready." Angular fills the blanks with actual data.

```
template: `
  <h1>{{ title.name }}</h1>           <!-- blank filled with title.name -->
  <button (click)="buy()">Buy</button> <!-- click event hooked up -->
`
```

You can have an **external** template file (`component.html`), but StreamCast uses **inline** templates because each component is small enough to keep in one file.

9 Data Binding — The 4 Types

This is the heart of Angular. Without binding, you'd update the DOM manually like in 1999.

9a. Interpolation `{{ ... }}` — TypeScript → DOM

Simple words: Print a value from your class into the HTML.

Analogy: Like the **mail-merge feature** in Word — "{Customer Name}" gets replaced with "John Smith".

```
<div>Welcome, {{ user.email }}</div>
<span>Items: {{ items.length }}</span>
```

9b. Property Binding `[prop]="..."` — TypeScript → DOM

Simple words: Set a DOM property (or component input) dynamically.

Analogy: Like a **thermostat dial** in your TypeScript controlling a **heater's temperature** in the DOM.

```
<button [disabled]="form.invalid">Save</button>
<img [src]="user.avatarUrl" />
<input [type]="show() ? 'text' : 'password'" /> <!-- from LoginComponent -->
```

9c. Event Binding `(event)="handler()"` — DOM → TypeScript

Simple words: When the user does something (click, type), call a method.

Analogy: Like a **doorbell** — the visitor presses (DOM event), the bell calls you (TypeScript method).

```
<button (click)="openCreate()">+ New title</button>
<input (input)="query.set($event.target.value)" />
<form (ngSubmit)="submit()">...</form>
```

9d. Two-way Binding `[(ngModel)]="x"` — Both directions

Simple words: Property + event combined. When user types, the variable updates AND when the variable changes, the input updates.

Analogy: Like a **shared Google Doc** — you type, your colleague sees. Your colleague edits, you see.

```
<input [(ngModel)]="searchText" />
<p>You typed: {{ searchText }}</p>
```

StreamCast prefers **Reactive Forms** over `ngModel`, but the concept is everywhere.

10 Directives

Simple words: Magic attributes you put on HTML to change its behavior.

Analogy: Directives are **road signs** — same road (HTML element), but signs tell you "STOP", "LOOP HERE", "DETOUR".

Two flavors:

10a. Structural directives (start with `*`) — change the DOM tree

`*ngIf` — conditional rendering

```
<div *ngIf="loading()">Loading spinner...</div>
<button *ngIf="canEdit()">Edit</button>      <!-- from titles.component -->
```

Analogy: A **light switch** — element exists only when condition is true.

`*ngFor` — repeat for each item

```
<li *ngFor="let t of titles; trackBy: trackId">{{ t.name }}</li>
```

Analogy: A **photocopier** — make one copy per row in the array. The `trackBy` part is important — it's like giving each photocopy a **serial number** so the copier doesn't reprint identical pages.

`*ngSwitch` — multiple branches

```
<div [ngSwitch]="status">
  <span *ngSwitchCase="'ACTIVE'">Live</span>
  <span *ngSwitchCase="'EXPIRED'">Done</span>
  <span *ngSwitchDefault>Unknown</span>
</div>
```

10b. Attribute directives — change appearance/behavior of existing elements

`ngClass` — toggle CSS classes

```
<span [ngClass]="{ 'sc-chip-success': t.status === 'AVAILABLE',
                  'sc-chip-warn':    t.status === 'DRAFT' }">
  {{ t.status }}
</span>
```

Analogy: Like **highlighter colors** — yellow for important, pink for urgent. Same text, different highlight based on category.

`ngStyle` — inline styles dynamically

```
<div [ngStyle]="{ background: poster(t) }"></div>
```

11 Pipes

Simple words: Transform data right inside the template without changing the original.

Analogy: Pipes are **filters on a camera** — the photo (data) is the same, but the filter (pipe) changes how it looks. Black-and-white, sepia, etc.

Built-in pipes:

```
{{ releaseDate | date:'mediumDate' }}      <!-- "May 19, 2026" -->
{{ price | currency:'USD' }}               <!-- "$42.50" -->
{{ name | uppercase }}                    <!-- "AFRIN" -->
{{ description | slice:0:100 }}             <!-- first 100 chars -->
{{ items | json }}                        <!-- pretty-print object -->
```

Custom pipe example:

```
@Pipe({ name: 'truncate', standalone: true })
export class TruncatePipe implements PipeTransform {
  transform(value: string, max = 50): string {
    return value.length > max ? value.slice(0, max) + '...' : value;
  }
}

// usage
{{ longTitle | truncate:30 }}
```

Pure vs impure pipes:

- **Pure** (default): runs only when input reference changes. Fast.
- **Impure**: runs on every change-detection tick. Use sparingly (e.g., live clock).

Part 2 — Architecture Concepts

12 Services

Simple words: Plain classes that hold logic or talk to APIs. Components use them.

Analogy: Components are **waiters** at a restaurant — they greet, take orders, deliver food. Services are the **kitchen** — they actually cook the food.

Why separate? A waiter shouldn't need to know how to bake bread. A `LoginComponent` shouldn't need to know how JWT decoding works.

```
// titles.service.ts – kitchen
@Injectable({ providedIn: 'root' })
export class TitlesService {
  private http = inject(HttpClient);
  list(): Observable<Title[]> {
    return this.http.get<Title[]>(`${environment.apiBase}/api/titles`);
  }
}

// titles.component.ts – waiter
export class TitlesComponent {
  private api = inject(TitlesService);
  refresh() {
    this.api.list().subscribe(rows => this.rows.set(rows));
  }
}
```

13 Dependency Injection (DI)

Simple words: You declare what your class needs; Angular automatically gives it to you.

Analogy: Imagine you build a car. You don't manufacture your own tires — the supplier delivers them. DI is the **supplier system**: you say "I need tires" (a service), Angular hands them over.

Two styles in StreamCast:

```
// Old way: constructor injection
constructor(private api: TitlesService, private router: Router) {}

// New way: inject() function (preferred in standalone)
private api = inject(TitlesService);
private router = inject(Router);
```

Why DI matters:

1. **Testability** — in tests, you can swap the real `TitlesService` for a fake one.
2. **Singletons** — only one `AuthService` instance exists in the whole app, so user state is shared.
3. **No new everywhere** — you don't manually construct services with all their dependencies.

`@Injectable({ providedIn: 'root' })` means "create one instance, available everywhere".

14 Lifecycle Hooks

Simple words: Special methods Angular calls at specific moments in a component's life.

Analogy: A component is like a **plant**. Lifecycle hooks are the moments you tend to it:

- **ngOnInit** — when the seed sprouts (first appears). Fetch data here.
- **ngOnChanges** — when you water it (input changes). React to new values.
- **ngAfterViewInit** — when all leaves are out (child views are ready). Touch the DOM here.
- **ngOnDestroy** — when you compost it (component removed). Clean up subscriptions.

```
export class TitlesComponent implements OnInit, OnDestroy {
  ngOnInit() {
    this.refresh();    // good place for initial API call
  }
  ngOnDestroy() {
    this.subscription?.unsubscribe();  // cleanup
  }
}
```

StreamCast often does init in the constructor for tiny components (since `inject()` is available there now), but for new components, prefer `ngOnInit`.

15 Routing

Simple words: Match URLs to components. No page reload.

Analogy: Routing is the **map app on your phone**. You type an address (URL); it shows you which area (component) to display.

```
export const routes: Routes = [
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' },
  { path: 'login', loadComponent: () => import('...').then(m => m.LoginComponent) },
  {
    path: '',
    canActivate: [authGuard],
    loadComponent: () => import('...').then(m => m.ShellComponent),
    children: [
      { path: 'dashboard', loadComponent: ... },
      { path: 'titles', loadComponent: ..., canActivate: [roleGuard([...])] },
      { path: 'titles/:id', loadComponent: ... },
    ]
  },
  { path: '**', redirectTo: 'dashboard' } // wildcard 404
];
```

Key parts:

- `path: 'login'` — URL pattern.
- `loadComponent` — lazy-load (download only when needed).
- `canActivate: [authGuard]` — guard runs before activation.
- `children` — nested routes (shown inside parent's `<router-outlet>`).
- `**` — catch-all (anything not matched).

Navigation

```
// programmatic
this.router.navigate(['/titles', t.id]); // → /titles/42

// in template
<a routerLink="/titles">Titles</a>
<a [routerLink]="['/titles', t.id]">{{ t.name }}</a>
```

`<router-outlet />`

The placeholder where the matched component renders. StreamCast's `AppComponent` is literally just:

```
template: `<router-outlet />`
```

16 Route Parameters

Simple words: Variables inside the URL.

Analogy: Like a **hotel room number** in your booking confirmation — same hotel (route), different room (id).

```
// route: { path: 'titles/:id', loadComponent: ... }  
  
// in TitleDetailComponent  
const route = inject(ActivatedRoute);  
const id = +route.snapshot.paramMap.get('id')!; // → 42
```

With `withComponentInputBinding()` (used in `app.config.ts`), you can even bind URL params straight to `@Input()`:

```
@Input() id!: string; // auto-filled from /titles/:id
```

17 Lazy Loading

Simple words: Download a chunk of code only when the user actually navigates there.

Analogy: Like **on-demand video streaming** — you don't download the whole 8-season show at startup; you fetch each episode when you click Play.

Why? Without lazy loading, the initial JS bundle would include all 13 feature pages → slow first load. StreamCast lazy-loads every route, so the first download is tiny.

```
{  
  path: 'usage',  
  loadComponent: () => import('./features/usage/usage.component').then(m => m.UsageComponent)  
}
```

Open DevTools → Network tab. Navigate to `/usage`. You'll see a new JS chunk download just then.

18 Guards

Simple words: Functions that say yes/no to a route navigation.

Analogy: Guards are **bouncers at a club**. You can't enter unless you meet the criteria — age (logged in), VIP list (role), dress code, etc.

```
// auth.guard.ts
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  if (auth.isAuthenticated()) return true;
  router.navigate(['/login']);
  return false;
};

export const roleGuard = (roles: Role[]): CanActivateFn => () => {
  const auth = inject(AuthService);
  if (!auth.isAuthenticated()) { /* redirect to login */ return false; }
  if (auth.hasAnyRole(roles)) return true;
  inject(Router).navigate(['/forbidden']);
  return false;
};
```

`roleGuard` is a **factory** — you pass the allowed roles, you get back a guard configured for those roles. Real-world parallel: a wedding's "VIP entrance" bouncer is configured with the actual guest list before the event starts.

Types of guards:

- `CanActivate` — can you enter?
- `CanActivateChild` — can you enter a child route?
- `CanDeactivate` — can you leave? (e.g., "you have unsaved changes")
- `Resolve` — pre-fetch data before component opens.
- `CanMatch` — should this route even be considered?

19 HttpClient

Simple words: Angular's tool for making HTTP requests to a backend.

Analogy: Like the **phone** in your office — you pick it up, dial, wait, get an answer.

```
private http = inject(HttpClient);

this.http.get<Title[]>('/api/titles') // GET
this.http.post<Title>('/api/titles', newTitle) // POST
this.http.put<Title>(`/api/titles/${id}`, t) // PUT
this.http.delete(`/api/titles/${id}`) // DELETE
```

The generic `<Title[]>` is just a TypeScript hint — the server sends JSON; you tell TS the shape so autocomplete works.

Why not just `fetch()`?

- HttpClient returns **Observables** (cancellable, composable).
- Automatic JSON parsing.
- **Interceptors** can transform every request.

20 Observables

Simple words: A stream of values over time. You **subscribe** to receive them.

Analogy: An Observable is like a **newsletter**. You subscribe → emails (values) arrive. Unsubscribe → no more emails. The newsletter doesn't write itself if no one subscribes (lazy).

```
const titles$ = this.api.list(); // Observable created, nothing fires yet

titles$.subscribe({           // NOW the HTTP request fires
  next: rows => console.log('got rows', rows),
  error: err => console.error(err),
  complete: () => console.log('done')
});
```

Observable vs Promise:

	Promise	Observable
Values	One	Many over time
Eager/lazy	Eager (fires immediately)	Lazy (fires on subscribe)
Cancellable	No	Yes (unsubscribe)
Operators	Few	100+ via RxJS

For HTTP, both work — but Observables let you do things like **cancel a stale search request** when the user types faster than the server responds.

21 RxJS Operators

Simple words: Functions that transform / filter / combine observable streams.

Analogy: Operators are **factory machines on an assembly line**. The raw material (values) enters; each machine (operator) modifies it; the finished product exits.

```

this.http.get<ApiResponse<Title[]>>('/contracts')
  .pipe(
    map(r => r.data),           // unwrap envelope
    tap(items => console.log(items)), // side-effect (logging)
    catchError(err => of([]))    // on error, give empty array
  );

```

The 6 you'll use 90% of the time:

Operator	Real-world analogy
<code>map</code>	A translator — converts each value to a new shape.
<code>filter</code>	A bouncer — only lets some values through.
<code>tap</code>	A CCTV camera — watches without changing the value.
<code>catchError</code>	Insurance — recover from errors.
<code>switchMap</code>	A TV remote — when you press a new channel, the previous one cuts off. Used for search-as-you-type.
<code>mergeMap</code>	A multi-line phone system — handles many calls in parallel.

22 Subjects (advanced Observables)

Simple words: Observables that you can also push values into manually.

Analogy: Regular Observable = **radio broadcast** you receive. Subject = a **microphone** you can speak into AND others listen.

```

const messages = new Subject<string>();
messages.subscribe(m => console.log('A:', m));
messages.subscribe(m => console.log('B:', m));
messages.next('hello'); // → "A: hello", "B: hello"

```

BehaviorSubject — same but holds a **current value** and emits it to new subscribers immediately. Used for global state pre-Signals.

Part 3 — Forms

23 Forms — Reactive vs Template-driven

Two ways to build forms in Angular.

Template-driven (small forms, defined in HTML)

```
<form #f="ngForm" (ngSubmit)="save(f.value)">
  <input name="email" [(ngModel)]="email" required email />
</form>
```

Reactive (defined in TypeScript — what StreamCast uses)

```
form = this.fb.nonNullable.group({
  email: ['', [Validators.required, Validators.email]],
  password: ['', Validators.required]
});
```

```
<form [formGroup]="form" (ngSubmit)="submit()">
  <input formControlName="email" />
  <input formControlName="password" type="password" />
  <button [disabled]="form.invalid">Sign in</button>
</form>
```

Analogy:

- Template-driven = **handwriting on a paper form** — quick for simple stuff.
- Reactive = **filling a database record** — structured, validated, programmatic.

Why reactive wins for big forms:

- All logic in TypeScript → testable.
- Dynamic forms (add/remove fields) are easy.
- Strong typing.

24 Validators

Simple words: Functions that check if a field is valid.

```
email: ['', [Validators.required, Validators.email, Validators.minLength(5)]]
```

Built-in: `required`, `email`, `min`, `max`, `minLength`, `maxLength`, `pattern`, `requiredTrue`.

Custom validator example:

```
function passwordStrong(c: AbstractControl): ValidationErrors | null {
  const v = c.value as string;
  return v.length >= 8 && /[A-Z]/.test(v) && /\d/.test(v)
    ? null
    : { weak: true };
}

password: ['', [Validators.required, passwordStrong]]
```

Then in template:

```
<div *ngIf="form.get('password')?.errors?.['weak']">
  Password must be 8+ chars with uppercase and digit.
</div>
```

Part 4 — Component Communication

25 @Input — Parent → Child

Simple words: A property that receives data from the parent component.

Analogy: Like **sending mail to your kid at college**. Parent sends → child receives.

```
// child
@Component({ selector: 'sc-status-chip', ... })
export class StatusChip {
  @Input() status: 'ACTIVE' | 'EXPIRED' = 'ACTIVE';
}

// parent
<sc-status-chip [status]="contract.status"></sc-status-chip>
```

26 @Output + EventEmitter — Child → Parent

Simple words: A property the child uses to emit events the parent listens to.

Analogy: Like the **kid at college sending you their grade report**. Child emits → parent reacts.

```
// child
@Output() saved = new EventEmitter<Title>();
onSave(value: Title) { this.saved.emit(value); }

// parent
<sc-title-form (saved)="refresh($event)"></sc-title-form>
```

`$event` is whatever the child passed to `emit()`.

27 Shared Service — Sibling Communication

Problem: Two components that aren't parent/child need to share state. They can't @Input/@Output each other.

Solution: Put the state in a service. Both components inject it.

Analogy: Two siblings can't talk through their dad's phone, but they both have access to the **family WhatsApp group** (service) — they post there, everyone sees.

```

@Injectables({ providedIn: 'root' })
export class AuthService {
  private _user = signal<CurrentUser | null>(null);
  user = this._user.asReadonly();
  setUser(u: CurrentUser) { this._user.set(u); }
}

// LoginComponent writes
this.auth.setUser(newUser);

// ShellComponent reads – auto re-renders
template: `<span>{{ auth.user()?.email }}</span>`

```

28 ng-content (Content Projection)

Simple words: A slot where the parent can put markup that the child component renders inside itself.

Analogy: Like a **picture frame**. The frame (child component) has an empty middle (`<ng-content>`). You hang any photo (parent's markup) inside it.

```

// child component
@Component({
  selector: 'sc-card',
  template: `
    <div class="sc-card">
      <ng-content></ng-content>      <!-- slot -->
    </div>
  `
})
export class CardComponent {}

// parent uses it
<sc-card>
  <h2>Hello</h2>
  <p>Anything I put here lands inside the card.</p>
</sc-card>

```

Multi-slot:

```

template: `
  <header><ng-content select="[header]"></ng-content></header>
  <main><ng-content></ng-content></main>
`

```

29 ng-template & ng-container

`<ng-template>` — a template fragment that **doesn't render unless referenced**. Used with `*ngIf else :`

```
<div *ngIf="user; else loading">Welcome {{ user.email }}</div>
<ng-template #loading>Loading...</ng-template>
```

`<ng-container>` — a wrapper that doesn't produce any DOM. Useful when you need a structural directive on multiple sibling elements without adding a `<div>`:

```
<ng-container *ngIf="show">
  <h1>Title</h1>
  <p>Body</p>
</ng-container>
```

Part 5 — Modern Angular (StreamCast uses all of these)

30 Signals (Angular 17 Reactivity)

Simple words: A value holder that tells everyone watching when it changes.

Analogy: A signal is like a **price tag in a supermarket**. When the price changes, every shelf-sticker, every checkout system, every app instantly sees the new price.

```
const count = signal(0);
console.log(count());      // read with () → 0
count.set(5);              // write
count.update(n => n + 1);   // update based on previous
```

StreamCast example:

```
export class TitlesComponent {
  rows    = signal<Title[]>([]);
  loading = signal(false);
  query   = signal('');

  // computed = derived signal, auto-recomputes when inputs change
  filtered = computed(() => {
    const q = this.query().trim().toLowerCase();
    return q ? this.rows().filter(t => t.name.toLowerCase().includes(q))
              : this.rows();
  });
}
```

In the template:

```
<input [value]="query()" (input)="query.set($event.target.value)" />
<div *ngFor="let t of filtered()">{{ t.name }}</div>
```

When the user types, `query` updates → `filtered` automatically recomputes → template refreshes. **No subscriptions, no manual change detection.**

31 computed() — Derived Signals

Simple words: A signal whose value is calculated from other signals. Auto-updates.

Analogy: Like a **spreadsheet formula**. Cells A1 and B1 hold numbers; C1 = `=A1+B1`. Change A1 → C1 updates automatically.

```
canEdit = computed(() => this.auth.hasAnyRole(['ADMIN', 'CONTENT_OWNER']));
```

`canEdit()` updates whenever `auth.role()` changes, **without** you wiring anything.

32 effect() — Side Effects on Signal Changes

Simple words: Run code whenever the signals you read inside change.

Analogy: Like setting up a **CCTV motion sensor**. Whenever motion (signal change) occurs, the lights turn on (effect runs).

```
effect(() => {  
  console.log('user is now', this.auth.user());  
});
```

Used sparingly — usually for side effects like logging or syncing to localStorage.

33 Interceptors

Simple words: Middleware that runs on every HTTP request.

Analogy: A **toll booth** on the highway. Every car (HTTP request) must pass through. The booth (interceptor) can attach a sticker (JWT header), check for issues, or send the car back.

StreamCast's `authInterceptor` :

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const token = auth.user()?.token;

  // Add Authorization header
  const cloned = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;

  // Handle 401 globally
  return next(cloned).pipe(
    catchError(err => {
      if (err.status === 401) auth.logout();
      return throwError(() => err);
    })
  );
};
```

Installed in `app.config.ts` :

```
provideHttpClient(withInterceptors([authInterceptor]))
```

Now every HTTP call goes through this — you never write `Authorization: Bearer ...` manually in any service. Power of DRY (Don't Repeat Yourself).

34 Change Detection (CD)

Simple words: Angular's process of looking at your data and updating the DOM.

Analogy: Imagine a **museum guide doing rounds every X seconds**, checking each exhibit (component) to see if the description (bound value) changed. If yes, repaint the placard.

Two strategies:

Default (`ChangeDetectionStrategy.Default`)

Checks every component on every "tick" (after every async event). Safe but slow for huge UIs.

OnPush

Only checks when:

1. An `@Input()` changes by reference, OR
2. An event fires inside the component, OR
3. A Signal it reads changes, OR

4. An async pipe emits.

Much faster. With Signals, you get fine-grained reactivity automatically.

```
@Component({  
  changeDetection: ChangeDetectionStrategy.OnPush,  
  ...  
})
```

35 The async pipe

Simple words: Subscribes to an Observable in the template and auto-unsubscribes when the component is destroyed.

Analogy: Hiring a **personal mail-handler** who opens your subscription letters (Observable) and shows you the content, then politely cancels your subscriptions when you move out.

```
titles$ = this.api.list();
```

```
<div *ngFor="let t of titles$ | async">{{ t.name }}</div>
```

No `subscribe()`, no `ngOnDestroy` cleanup. Eliminates memory leaks.

36 Environment Configuration

Simple words: Different values for dev vs prod, set at build time.

Analogy: Like having **dev keys and prod keys** in your wallet — same shape, different doors.

```
// environments/environment.ts  
export const environment = { production: false, apiBase: 'http://localhost:8082' };  
  
// environments/environment.prod.ts  
export const environment = { production: true, apiBase: 'https://api.streamcast.io' };
```

Used in services:

```
this.http.get(`${environment.apiBase}/api/titles`)
```

At build time, Angular swaps which file is bundled based on the configuration.

37 AOT vs JIT

JIT (Just-In-Time): Angular compiles your templates **in the browser** at startup. Bigger bundle, slower start. (Old default.)

AOT (Ahead-Of-Time): Angular compiles templates **at build time** on your machine. Smaller bundle, faster start. (Modern default.)

Analogy: JIT = ordering a **made-to-order meal** at the restaurant (wait at the table). AOT = a **frozen meal** prepared in advance (instant heat-and-serve).

Production builds (`ng build`) use AOT.

38 Tree-shaking

Simple words: The bundler throws away code you never use.

Analogy: Like **only packing the clothes you wear** for a trip. Don't bring the 50 t-shirts you'll never touch.

If `TitlesService` is never imported, it won't be in the final JS bundle. This is why `@Injectable({ providedIn: 'root' })` is preferred over module-based providers — tree-shakable.

39 The inject() function

Simple words: A new way to get dependencies without a constructor.

Analogy: Old style = **standing in line at a counter** to collect parcels. `inject()` = **vending machine** — press button anywhere, get item.

```
// Old
constructor(private http: HttpClient, private router: Router) {}

// New
private http = inject(HttpClient);
private router = inject(Router);
```

Required in functional guards/interceptors (which have no class/constructor):

```
export const authGuard: CanActivateFn = () => {  
  const auth = inject(AuthService); // only way here  
  return auth.isAuthenticated();  
};
```

40 Provider Functions (provideX)

Simple words: Modern way to register app-wide things in `app.config.ts`.

```
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideRouter(routes, withComponentInputBinding()),  
    provideHttpClient(withInterceptors([authInterceptor])),  
    provideAnimations(),  
    importProvidersFrom(MatSnackBarModule) // bridge old NgModule-style  
  ]  
};
```

Analogy: Like **plugging appliances into the power strip** at the heart of your home. One central place.

Old way needed an `AppModule` with `BrowserModule`, `HttpClientModule`, `RouterModule.forRoot(routes)` — verbose and harder to tree-shake.

Concepts Working Together — One Click, 19 Concepts

Let me trace one user click through every concept to show how they connect.

User clicks "Sign in" in `LoginComponent` :

1. **Event binding** `(ngSubmit)="submit()"` calls the method.
2. The method reads the **Reactive Form** value.
3. **Service injection** — `this.auth` was provided by **DI**.
4. `auth.login(creds)` calls **HttpClient** → returns an **Observable**.
5. `.pipe(tap(...))` — **RxJS operator** stores the JWT.
6. `.subscribe()` — fires the network call.
7. The **HTTP Interceptor** receives the request, but for `/auth/login` there's no token yet so it just forwards.
8. Backend responds.
9. `tap` runs — calls `persist()` which decodes the **JWT**, builds a `CurrentUser`, calls `signal.set(user)`.
10. Every **computed signal** that reads `_user` (`isAuthenticated`, `role`) recomputes.
11. Every template that reads them (Shell nav, guards) re-renders.
12. The `next` handler calls `router.navigate(['/dashboard'])`.
13. **authGuard** runs → `isAuthenticated()` is now true → passes.
14. **roleGuard** for that route runs → role matches → passes.
15. The **lazy-loaded** chunk for `DashboardComponent` downloads if it hasn't already.
16. Dashboard mounts → **lifecycle hook** `ngOnInit` (or constructor) runs.
17. **Computed signal** `tiles = computed(() => allTiles.filter(t => auth.hasAnyRole(t.roles)))` filters to allowed tiles.
18. Template renders tiles via `*ngFor` + `trackBy`.
19. Each tile is a `routerLink` to a feature.

Every concept in this guide fired in those 19 steps. That's why learning Angular feels overwhelming at first — you can't really learn one piece without the others.

How to Internalize This Fast

1. **Open** `streamcast-ui/src/app/features/auth/login.component.ts` **and trace what every annotation, function, and binding does.** Re-read this guide while looking at the code.
2. **Change one thing and see what breaks.** Remove `[disabled]="form.invalid"` → button is always clickable. Remove `(ngSubmit)="submit()"` → form does nothing. This builds intuition faster than any tutorial.
3. **Build a minimal copy.** Make a tiny "TodoList" component with: a signal for items, ngFor to display, ngIf for empty state, a form to add, a service to fetch from JSONPlaceholder. You'll use ~20 of the concepts above in 100 lines of code.

You're set. Keep this PDF open next to your code editor. When you encounter a concept in the StreamCast source, jump to its section. After ~3 reads you'll have the whole map in your head and the project will feel small.

One-page summary of the 40 concepts

#	Concept	One-line description
1	Angular	Toolkit for SPA web apps
2	SPA	One HTML page, JS swaps content
3	TypeScript	JS + static types
4	Decorators	@Annotations adding metadata
5	Component	Reusable UI piece (class + template + style)
6	Selector	Custom HTML tag the component matches
7	Standalone	Component declares own imports, no NgModule
8	Template	HTML with Angular syntax
9	Data binding	{{}} [prop] (event) [(two-way)]
10	Directives	*ngIf, *ngFor, ngClass, ngStyle...
11	Pipes	Template formatters (date, currency...)
12	Services	Reusable classes for logic / HTTP
13	DI	Framework supplies dependencies
14	Lifecycle hooks	ngOnInit, ngOnDestroy, etc.
15	Routing	URL → component mapping
16	Route params	Values in URL (/titles/:id)
17	Lazy loading	Download chunks on demand
18	Guards	Yes/no decisions on navigation
19	HttpClient	Typed HTTP wrapper, returns Observables
20	Observables	Lazy async streams
21	RxJS operators	map, tap, catchError, switchMap...
22	Subjects	Push-able Observables
23	Forms	Reactive (TS) or Template-driven (HTML)
24	Validators	Required, email, custom rules
25	@Input	Parent → child data
26	@Output	Child → parent events
27	Shared service	Sibling state sharing

28	ng-content	Content projection slot
29	ng-template/container	Inert template / DOM-less wrapper
30	Signals	Reactive value holder (Angular 17)
31	computed()	Derived signal, auto-updates
32	effect()	Side effect on signal change
33	Interceptors	Middleware on every HTTP request
34	Change detection	Process that updates DOM
35	async pipe	Subscribes in template, auto-cleans
36	Environment	Build-time config (dev/prod)
37	AOT vs JIT	Compile at build vs in browser
38	Tree-shaking	Drop unused code from bundle
39	inject()	Get dependency outside constructor
40	provideX	Standalone way to register providers